

Lembar Materi & Praktikum: Environment Management, Server Deployment, dan Systemd Service pada Golang Gin

Durasi: 3 Jam (180 Menit) **Level:** Intermediate (Menengah)

Praktikum ini melengkapi materi teori pada [17. multi-env-api.md](#). Anda akan dipandu untuk mempersiapkan aspek operasional tingkat produksi (*production-ready operations*) pada proyek **Sistem Tiket Konser** (`go-tiket-konser`) menggunakan framework **Gin** dan **GORM**. Kita akan belajar memisahkan konfigurasi sensitif dari kode sumber menggunakan manajemen **Environment Variables** (`.env`), mendesain konfigurasi terpusat tipe-kuat (*strongly-typed*), melakukan kompilasi silang (*cross-compilation*) untuk server Linux, serta mendaftarkan aplikasi backend sebagai daemon background menggunakan **Linux Systemd Service**.

Sesi 1: Teori Environment Management & Config Centralization (45 Menit)

1. Mengapa Environment Variables Sangat Penting?

Salah satu kesalahan keamanan paling fatal (*critical security flaw*) yang sering dilakukan developer pemula adalah menulis langsung (*hardcoding*) kredensial rahasia ke dalam kode program, seperti:

- Kunci rahasia JWT (`JWT_SECRET`)
- Password database PostgreSQL
- API Token pihak ketiga (seperti Mailgun atau Cloudinary)

Jika kode sumber tersebut diunggah ke repositori publik (seperti GitHub), pihak tidak bertanggung jawab dapat memindai repositori tersebut menggunakan bot otomatis untuk meretas database Anda dalam hitungan menit.

Solusi: Gunakan manajemen variabel lingkungan (*Environment Variables*). Kita menyimpan konfigurasi dinamis ini pada sebuah berkas rahasia `.env` di lingkungan server lokal, dan mengabaikannya dari git agar tidak ikut terunggah.

2. Tiga Lingkungan Pengembangan (Environments)

Dalam siklus hidup rekayasa perangkat lunak, sistem minimal berjalan pada 3 lingkungan terpisah:

1. **Development**: Komputer lokal developer. Menggunakan database lokal (atau bahkan SQLite), mode debug aktif, dan logging verbose.
2. **Staging (Testing)**: Server awan tiruan tempat tim penjamin mutu (QA) atau klien melakukan uji coba fitur sebelum dirilis secara global.
3. **Production (Live)**: Server asli yang melayani pengguna riil. Kredensial di sini sangat rahasia, database berskala besar, mode Gin diatur ke `release` (tanpa log debugging berwarna untuk menghemat I/O disk).

3. Centralized Configuration Struct (Pola Clean Code)

Meskipun kita bisa memanggil `os.Getenv("VARIABLE_NAME")` langsung di sembarang tempat, cara tersebut membuat kode kita berantakan, sulit dipelihara, dan rentan salah ketik (*typo*). Pola terbaik (*best practice*) adalah memuat seluruh variabel `.env` di awal aplikasi menyala ke dalam sebuah **Struct Konfigurasi Terpusat** (`AppConfig`). Ini memfasilitasi konversi tipe data (misal string dari env diubah menjadi integer untuk Port, atau boolean untuk Debug Mode).

Sesi 2: Teori Deployment Server & Cross-Compilation di Go (45 Menit)

1. Keunggulan Kompilasi Go

Berbeda dengan Node.js, Python, atau Java yang membutuhkan mesin virtual (*runtime interpreter*) di server produksi (seperti Node, Python, atau JVM), compiler Go mengompilasi seluruh kode program beserta dependencies-nya menjadi satu berkas biner tunggal (*single standalone binary machine code*).

Di server produksi, kita cukup menjalankan berkas biner ini saja tanpa perlu menginstal compiler Go atau folder `node_modules` raksasa. Hal ini membuat aplikasi Go sangat ringan, hemat memori, dan sangat cepat saat dinyalakan.

2. Cross-Compilation Lintas Platform

Sebagian besar developer menulis kode di Windows atau macOS, tetapi server produksi web biasanya berjalan di Linux (seperti Ubuntu Server atau Debian). Go mempermudah hal ini dengan fitur **Cross-Compilation**. Kita bisa mengompilasi biner Linux dari komputer Windows kita hanya dengan menyuntikkan dua variabel lingkungan kueri compiler:

- `GOOS` : Sistem operasi target (misal: `"linux"` , `"windows"` , `"darwin"`).
- `GOARCH` : Arsitektur CPU target (misal: `"amd64"` untuk Intel/AMD 64-bit, `"arm64"` untuk Apple Silicon atau Raspberry Pi).

Perintah Kompilasi Silang (dari terminal Windows PowerShell):

```
$env:GOOS="linux"; $env:GOARCH="amd64"; go build -o go-tiket-konser-linux main.go
```

Setelah berkas biner Linux terbentuk, kita cukup mentransfer berkas `go-tiket-konser-linux` tersebut ke Virtual Private Server (VPS) target menggunakan protokol transfer file seperti SCP atau SFTP.

Sesi 3: Menjalankan Backend Sebagai Daemon dengan Linux Systemd (45 Menit)

1. Apa itu Daemon dan Systemd?

Jika Anda login ke VPS menggunakan koneksi SSH lalu menjalankan aplikasi dengan perintah `./go-tiket-konser-linux`, aplikasi tersebut akan langsung mati begitu Anda menutup terminal SSH Anda.

Untuk menjaga aplikasi backend tetap menyala secara terus-menerus di latar belakang (*background service*), kita harus mendaftarkannya sebagai **Daemon** (Background Service). Di sistem operasi Linux modern (seperti Ubuntu/Debian/CentOS), standard industri untuk mengelola daemon adalah menggunakan **Systemd**.

2. Struktur Berkas Systemd Service Unit

Kita membuat file konfigurasi unit service (misal: `/etc/systemd/system/go-tiket-konser.service`) dengan petunjuk instruksi berikut:

- `WorkingDirectory` : Lokasi folder aplikasi di server (tempat file `.env` berada).
- `ExecStart` : Perintah absolut untuk mengeksekusi biner backend.
- `Restart=always` : Menginstruksikan sistem operasi untuk otomatis menyalakan ulang aplikasi jika aplikasi crash atau server VPS melakukan reboot.
- `EnvironmentFile` : (Opsional) Mengisi variabel env langsung dari berkas tertentu.

3. Perintah Utama Manajemen Service (Systemctl)

- `sudo systemctl daemon-reload` : Memperbarui konfigurasi systemd setelah kita mengedit berkas `.service`.
- `sudo systemctl start go-tiket-konser` : Menyalakan service backend.
- `sudo systemctl status go-tiket-konser` : Memeriksa apakah service berjalan lancar atau mendeteksi error saat startup.

- `sudo systemctl enable go-tiket-konser` : Mengatur agar service otomatis menyala saat mesin server VPS baru dihidupkan (booting).
 - `journalctl -u go-tiket-konser -f` : Membuka *real-time stream* log dari aplikasi backend yang dikelola oleh systemd (memanfaatkan journald).
-

Sesi 4: Praktikum Mandiri – Konfigurasi Terpusat & Langkah Deployment (45 Menit)

Pelajari langkah-langkah di bawah ini untuk memahami implementasi arsitektur operasional terpusat.

1. Desain Config Struct Terpusat: [config/config.go](#)

Buat berkas baru bernama `config.go` di bawah folder `config/` untuk memusatkan pembacaan variabel lingkungan:

```
package config

import (
    "log"
    "os"
    "strconv"

    "github.com/joho/godotenv"
)

type AppConfig struct {
    AppPort      int
    AppMode      string
    DBUser       string
    DBPass       string
    DBHost       string
    DBPort       string
    DBName       string
    JWTSecret    string
    SMTPHost     string
    SMTPPort     string
    SenderEmail  string
    AuthPassword string
}

// LoadConfig memuat berkas .env dan merangkumnya ke dalam struct AppConfig dengan
// tipe data yang sesuai
func LoadConfig() AppConfig {
    // Memuat berkas .env. Jika tidak ada, log peringatan akan dicetak (berguna di
    // server produksi stateless)
```

```

    if err := godotenv.Load(); err != nil {
        log.Println("Peringatan: File .env tidak ditemukan, sistem akan
menggunakan env bawaan OS")
    }

    portStr := os.Getenv("APP_PORT")
    port, err := strconv.Atoi(portStr)
    if err != nil {
        port = 8080 // Default port jika kosong atau tidak valid
    }

    return AppConfig{
        AppPort:      port,
        AppMode:       os.Getenv("APP_MODE"),
        DBUser:        os.Getenv("DB_USER"),
        DBPass:        os.Getenv("DB_PASS"),
        DBHost:        os.Getenv("DB_HOST"),
        DBPort:        os.Getenv("DB_PORT"),
        DBName:        os.Getenv("DB_NAME"),
        JWTSecret:     os.Getenv("JWT_SECRET"),
        SMTPHost:      os.Getenv("SMTP_HOST"),
        SMTPPort:      os.Getenv("SMTP_PORT"),
        SenderEmail:   os.Getenv("SENDER_EMAIL"),
        AuthPassword: os.Getenv("AUTH_PASSWORD"),
    }
}

```

2. Modifikasi Koneksi Database: [config/db.go](#)

Buka berkas `config/db.go`, sesuaikan inisialisasi agar membaca parameter DSN Postgres secara dinamis dari `LoadConfig()`:

```

package config

import (
    "fmt"
    "log"

    "go-tiket-konser/models"
    "gorm.io/driver/postgres"
    "gorm.io/gorm"
)

var DB *gorm.DB

func InitDB() {
    // 1. Muat konfigurasi terpusat
    cfg := LoadConfig()

    // 2. Susun DSN Postgres dinamis dari Environment Variables

```

```

        dsn := fmt.Sprintf("host=%s user=%s password=%s dbname=%s port=%s
sslmode=disable TimeZone=Asia/Jakarta",
            cfg.DBHost,
            cfg.DBUser,
            cfg.DBPass,
            cfg.DBName,
            cfg.DBPort,
        )

        var err error
        DB, err = gorm.Open(postgres.Open(dsn), &gorm.Config{})
        if err != nil {
            log.Fatal("Gagal menghubungkan ke database: ", err)
        }

        // 3. Jalankan AutoMigrate
        err = DB.AutoMigrate(&models.Concert{}, &models.TicketCategory{},
            &models.Customer{},
            &models.Booking{}, &models.BookingDetail{}, &models.User{},
            &models.BlacklistedToken{})
        if err != nil {
            log.Fatal("Gagal menjalankan migrasi database: ", err)
        }

        log.Println("Koneksi database sukses dan migrasi selesai")
    }
}

```

3. File Main Integrasi: [main.go](#)

Buka file `main.go` di root proyek Anda, dan atur mode Gin serta port server sesuai konfigurasi:

```

package main

import (
    "fmt"
    "go-tiket-konser/config"
    "go-tiket-konser/routes"

    "github.com/gin-gonic/gin"
)

func main() {
    // 1. Muat konfigurasi awal
    cfg := config.LoadConfig()

    // 2. Set Gin mode berdasarkan env mode
    if cfg.AppMode == "production" {
        gin.SetMode(gin.ReleaseMode)
    }

    // 3. Inisialisasi Database
}

```

```

config.InitDB()

// 4. Setup Router dengan menyuntikkan konfigurasi AppConfig
r := routes.SetupRouter(config.DB, cfg)

// 5. Jalankan server
addr := fmt.Sprintf(":%d", cfg.AppPort)
fmt.Printf("⇒ Aplikasi berjalan di port %s dalam mode [%s]\n", addr,
cfg.AppMode)
_ = r.Run(addr)
}

```

3b. Integrasi Dynamic Email Service pada Router: [routes/routes.go](#)

Sesuaikan berkas `routes/routes.go` Anda untuk menerima konfigurasi `AppConfig` agar instansiasi `EmailService` dari Day 16 dimuat secara dinamis dari file `.env` :

```

package routes

import (
    "go-tiket-konser/config"
    "go-tiket-konser/handler"
    "go-tiket-konser/repository"
    "go-tiket-konser/service"

    "github.com/gin-gonic/gin"
    "gorm.io/gorm"
)

func SetupRouter(db *gorm.DB, cfg config.AppConfig) *gin.Engine {
    r := gin.Default()

    // 1. Inisiasi EmailService secara dinamis membaca konfigurasi dari Env
    emailService := service.NewEmailService(cfg.SMTPHost, cfg.SMTPPort,
cfg.SenderEmail, cfg.AuthPassword)

    // 2. Inisiasi service booking dan suntikkan emailService ke dalamnya
    bookingRepo := repository.NewBookingRepository(db)
    customerRepo := repository.NewCustomerRepository(db)
    bookingService := service.NewBookingService(db, bookingRepo, customerRepo,
emailService)
    bookingHandler := handler.NewBookingHandler(bookingService)

    // 3. Daftarkan rute booking
    api := r.Group("/api/v1")
    {
        api.POST("/bookings", bookingHandler.CreateBooking)
    }
}

```

```
    return r
}
```

4. Setup `.env.example` dan `.gitignore`

Buat file template bernama `.env.example` di root proyek Anda (dan **WAJIB** di-commit ke Git) agar developer lain tahu daftar kunci variabel yang diperlukan:

```
APP_PORT=8080
APP_MODE=development
DB_USER=postgres
DB_PASS=secret45
DB_HOST=127.0.0.1
DB_PORT=5434
DB_NAME=eticketdb
JWT_SECRET=MasukanKunci0torisasiRahasiaDisini123!

# SMTP Email Configuration
SMTP_HOST=smtp.mailtrap.io
SMTP_PORT=2525
SENDER_EMAIL=support@concertticket.com
AUTH_PASSWORD=your_smtp_password
```

Jangan lupa menambahkan berkas `.env` ke dalam file `.gitignore` agar kredensial Anda tidak bocor ke publik:

```
# Tambahkan baris ini pada berkas .gitignore Anda
.env
```

5. Template Konfigurasi Systemd Service di Server Ubuntu

Ketika mendeploy biner Linux di server VPS Ubuntu, kita meletakkan file biner di folder `/var/www/go-tiket-konser` beserta file `.env` produksi asli.

Buat file baru di VPS Anda di `/etc/systemd/system/go-tiket-konser.service` dengan perintah:

```
sudo nano /etc/systemd/system/go-tiket-konser.service
```

Isikan berkas dengan konfigurasi systemd service berikut:

```
[Unit]
Description=Backend API Go Sistem Tiket Konser
After=network.target

[Service]
```



```
Type=simple
User=ubuntu
WorkingDirectory=/var/www/go-tiket-konser
ExecStart=/var/www/go-tiket-konser/go-tiket-konser-linux
Restart=always
RestartSec=5s
StandardOutput=syslog
StandardError=syslog
SyslogIdentifier=go-tiket-konser

[Install]
WantedBy=multi-user.target
```

Setelah disimpan, aktifkan service di server Linux Anda dengan perintah berurutan:

```
# 1. Reload daemon agar membaca service baru
sudo systemctl daemon-reload

# 2. Nyalakan Service
sudo systemctl start go-tiket-konser

# 3. Aktifkan agar otomatis menyala saat server reboot
sudo systemctl enable go-tiket-konser

# 4. Periksa log secara real-time
journalctl -u go-tiket-konser -f
```

Sesi 5: Tantangan Praktikum Mandiri (Tugas Mandiri)

Selesaikan dua tantangan operasional berikut untuk melatih kemampuan administrasi sistem (*system administration*) dan penulisan skrip otomasi.

Tantangan 1: Memisahkan File Config DB Antara PostgreSQL (Production) dan SQLite (Development)

Dalam pengembangan lokal, seringkali setup PostgreSQL server dirasa berat atau memakan memori komputer lokal. Kita ingin aplikasi menggunakan **SQLite** secara lokal, namun tetap menggunakan **PostgreSQL** di server produksi.

- **Tugas:**

1. Buka berkas `dto/` atau `config/config.go`, tambahkan field baru `DBDriver` yang dibaca dari environment variable `DB_DRIVER` di file `.env`.
2. Modifikasi `config/db.go`. Jika nilai `DBDriver` adalah `"sqlite"`, inisialisasi GORM menggunakan driver SQLite:

```
import "gorm.io/driver/sqlite"
// ...
DB, err = gorm.Open(sqlite.Open("go-tiket-konser.db"), &gorm.Config{})
```

3. Jika nilainya "postgres", lakukan koneksi ke server PostgreSQL dengan DSN seperti biasa.
4. Uji coba keandalan switch database ini di komputer lokal Anda dengan mengubah isi `DB_DRIVER` pada file `.env`.

Tantangan 2: Otomatisasi Build Lintas Platform Menggunakan Makefile

Mengetik perintah cross-compilation yang panjang berulang kali sangat melelahkan dan rentan kesalahan ketik. Kita dapat memanfaatkan utilitas **Makefile** untuk merapikan alur kerja build lokal.

- **Tugas:**

1. Buat berkas bernama `Makefile` (tanpa ekstensi) di root folder proyek Anda.
2. Tulis perintah otomatisasi build target berikut:

```
.PHONY: build-windows build-linux clean run

APP_NAME=go-tiket-konser

# Build untuk Windows lokal
build-windows:
    go build -o bin/$(APP_NAME).exe main.go

# Cross-compile untuk Linux 64-bit
build-linux:
    env GOOS=linux GOARCH=amd64 go build -o bin/$(APP_NAME)-linux
    main.go

# Menghapus file biner hasil kompilasi
clean:
    rm -rf bin/

# Menjalankan aplikasi langsung
run:
    go run main.go
```

3. Uji coba utilitas ini dengan menjalankan perintah `make build-linux` atau `make build-windows` di terminal Anda.